

IPC

Inter-Process Communication

Az előadás anyaga

Dr. Vadász Dénes: Operációs rendszerek és
+ URL linkek alapján készült el

Témakörök

- Az IPC alapfogalmak és jellemzői
- IPC mechanizmusok:
 - Szignálok
 - Pipe
 - Üzenetsor
 - Osztott memória
 - Szemafor, Socket
- Mintapéldák

Processz élelciklusa - ismételés

A processzek életük során:

- *versenyeznek erőforrásokért:*
 - CPU-ért: *ütemezés, processz állapotok.*
 - Egyéb erőforrásokért: *erőforrás kezelés, holtpont probléma.*
- ***kommunikálnak: IPC mechanizmusok.***
- *együttműködnek: versenyhelyzet (konkurencia) probléma*

Alapfogalmak, elvek

„Több processzből álló alkalmazás esetén *alapvető mechanizmus az IPC (Inter-Process Communication, processzek közötti kommunikáció).*

- Multiprogramozású rendszerekben *abszolút természetes* eszköz.
- *Kooperáló processzek kommunikálnak, szinkronizálják futásukat.*

Alapfogalmak, elvek

A probléma kiindulási megfogalmazása:

- *processz legyen képes üzenetet küldeni másik processznek, processz legyen képes üzenetet fogadni más processztől.*
Hogyan?

A kiindulási megoldás - a rendszerhívások használata:

- legyen *send(message)* és *receive(message)* rendszerhívás.

A processzek között épüljön fel *kommunikációs kapcsolat* (link).

Inter-Process Communication

Már találkoztunk *IPC mechanizmussal*:

- *Szignálózás is IPC: kis információmennyiség átvitelével.*
- *A környezeti változókon kommunikálhatnak a szülő – gyermek viszonyban lévők (kis információmennyiség).*
- *Fájlokon keresztül is lehet (nagy mennyiség, de lassú).*

A kérdés az, hogy hogyan lehetne jó teljesítménnyel (gyorsan, sokat) információkat átadni?

Inter-Process Communication

„Inter-Process Communication (IPC):

Processzek közötti *kommunikációs mechanizmusok*.

Kommunikáció célja:

- információ csere,
- szinkronizáció.

Maga a kommunikáció is igényelhet szinkronizációt.

Kommunikációra példa a tananyagban: *signal* állapotátmenet megvalósítása egy *signal* kommunikációs mechanizmussal.

IPC - kommunikáció fajtái

IPC kommunikáció fajtái:

- *direkt kommunikáció, indirekt kommunikáció,*
- *egy-, kétirányú kommunikáció (aszimmetrikus, szimmetrikus),*
- *zéró, korlátozott kapacitású, végtelen kapacitású puffereelt,*
- *fix üzenethosszúságú, változó üzenethosszúságú,*
- *szinkron, aszinkron.*

IPC - Direkt kommunikáció

Direkt kommunikáció

A direkt kommunikáció esetén a kommunikáló processzeknek ismerniük kell egymást.

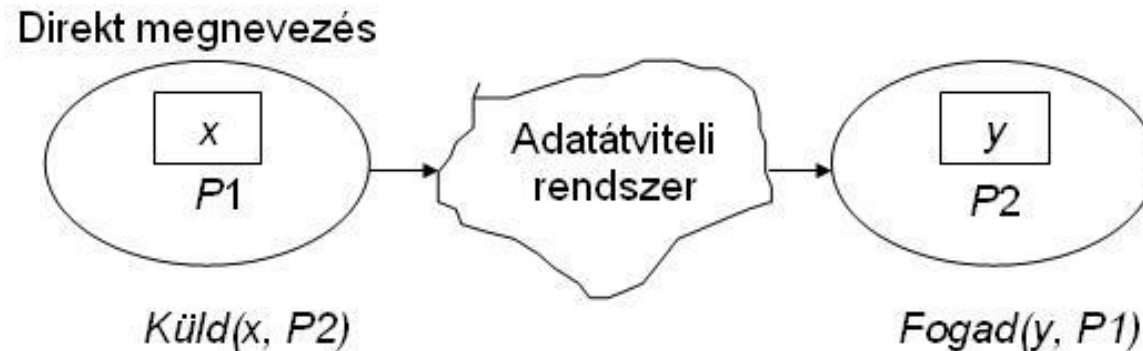
Többnyire két processz közötti kommunikáció.

- *Műveletei: küldés, fogadás (send, receive)*
(közben kiépül egy egyirányú- vagy kétirányú kapcsolat)
pl. signal

IPC - Direkt kommunikáció

A *direkt kommunikáció* két processz között zajlik, mind a *Küld*, mind a *Fogad* művelet megnevezi a *partner processzt*.

P1 elküldi a x változó értékét P2-nek, aki azt saját y változójába teszi el.



IPC - Indirekt kommunikáció

Indirekt kommunikáció

Az *indirekt kommunikáció* esetén a processzeknek egy közvetítő *entitást* kell ismerniük (üzenetsor, postaláda, stb.).

A kommunikációban résztvevőknek:

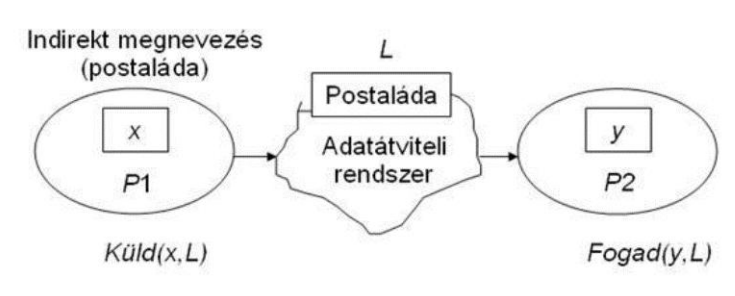
- a.) létre kell hozni az entitást vagy csatlakozni kell hozzá,*
- b.) információ küldés, fogadás,*
- c.) meg kell szüntetni az entitást.*

pl.: *fájlokon keresztüli kommunikáció, üzenetsor.*

IPC - Indirekt kommunikáció

Az *entitás* lehet:

- *Üzenetsor, mailbox* stb. objektum alkotja a kapcsolatot.
Ez önálló objektum, azonosítható.
- Ez lehet *csatorna*.
- Ezt a *közvetítő entitást* kell ismerni (asszociálni).



IPC – rendszerhívások – üzenetsor – előretekintés

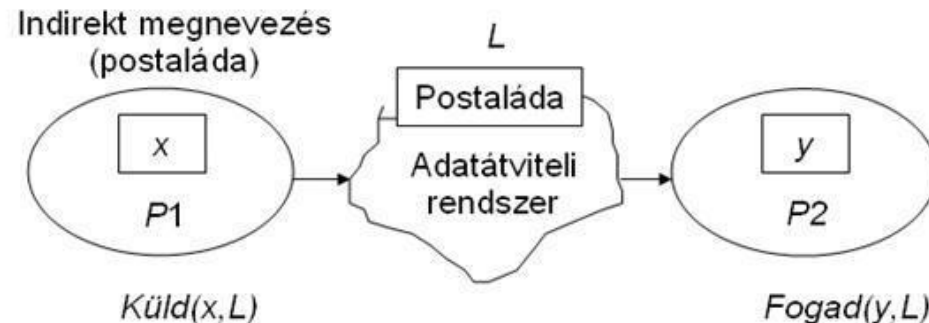
Szükséges (generikus) rendszerhívások:

- `create(msg-queue) ,`
- `destroy(msg-queue) ,`
- `send(msg-queue, message) ,`
- `receive(msg-queue, message) etc.`

IPC - Indirekt kommunikáció

Indirekt kommunikáció esetén a partnerek egy közvetítő objektumot (pl.: postaládát, vagy csatornát) hoznak létre.

A postaláda egy általában véges, de lehet korlátlan befogadóképességű FIFO tároló, amely a Küld-Fogad, (betesz-kivesz) műveletpárral kezelhető.



IPC - Indirekt kommunikáció

- A `Küld(<cím>, <postaláda>)` művelet a *üzenetet küld a postaláda szabad tárolóhelyére, azaz másolja.*
- Ha a postaláda tele van – akkor várakozik.
- A `Fogad(<cím>, <postaláda>)` művelet a *legrégebben várakozó üzenetet kimásolja a megadott saját címtartománybeli címre, helyét pedig felszabadítja.*

IPC - csatorna

A csatorna olyan kommunikációs objektum, amelyik két processzt kapcsol össze.

- *A csatorna lehet egyirányú (szimplex), osztottan kétirányú, azaz egyidejűleg egyirányú, de az irány változtatható (félduplex), vagy kétirányú (duplex).*
- *Tartalmazhat 0, véges vagy végtelen kapacitású átmeneti tárolót.*

IPC – csatorna: mailbox

Az üzeneteket *mailbox*, vagy *port mechanizmusokon* keresztül közvetítik.

- A *mailbox* (postaláda) az egy *objektum*, amibe a processz *üzeneteket tehet*, majd processz abból *üzeneteket vehet ki*.
- A *mailbox*-oknak van *azonosítójuk*.
- Ha két processz *osztozik közös postaládán* (mailbox-on, port-on) akkor *azon át válthatnak üzeneteket*.

IPC – mailbox, rendszerhívások – előrettekintés

A szükséges rendszerhívások:

- `create (mailbox)`
- `destroy (mailbox)`
- `share (mailbox)`
- `send (mailbox, message)`
- `receive (mailbox, message) etc.`

IPC – egy-kétirányú kommunikáció

Egy-, kétirányú kommunikáció

(aszimmetrikus, szimmetrikus)

- *Egyirányú: az egyik csak ad a másik csak vesz.*
- *Kétirányú: mindkét fél küld is fogad is.*

IPC - kapacitás sorok

Kommunikációnak három tárolási mód lehet: *zéró kapacitású sor*, *korlátozott kapacitású sor*, *végtelen kapacitású sor*.

a.) *Zéró kapacitású sor*: a kapcsolatnak *nincs puffere*, azaz a küldéssel egyidejűleg fogadni kell az üzenetet (szinkronizációt igényel) – *randevú szinkronizációnak* is nevezik.

b.) *Korlátozott kapacitású* a sor, maximum n *üzenetet* tárolhat átmenetileg a sor.

Inter-Process Communication

- Ha *teli van a sor*, a *küldőnek várakoznia* kell. Ha *üres*, a *fogadónak kell várakoznia*.
- *Van puffer*: A *küldés elválik a fogadástól*. A *küldéskor* egy *pufferbe kerül az információ*. A *fogadó innen veszi ki az elsőt*.

c.) *Végtelen kapacitású puffer.*

- *Végtelen kapacitású sor*: a *küldőnek sohasem kell várakoznia*.

IPC – szinkron, aszinkron

Szinkron/aszinkron küldés:

A küldő processz megvárja-e, hogy a fogadó megkapja az üzenetet.

Blokkolásos/nem blokkolásos (szinkron/aszinkron) fogadás:

- Ha mind a *küldés*, mind a *fogadás blokkolásos*, akkor a kommunikáció ***szinkron***.
- Ha *valamelyik (vagy mindkettő) nem blokkolásos*, akkor ***aszinkron***.

IPC mechanizmusok

- Fájlokon keresztüli kommunikáció.
- Vágólapon (Clipboard) keresztül.
- Az “environment“-en keresztüli kommunikáció.
- *Szignálózáson keresztüli kommunikáció.*
- *Csővezeték (pipe) névnélküli, nevesített csövek.*
- *A klasszikus message queue rendszer.*
- *Osztott memórián keresztüli kommunikáció.*

IPC mechanizmusok

- *Szemafor*, mint kommunikációs mechanizmusok.
- Socket kommunikáció.
- Remote Procedure Call.

IPC - fájlokon keresztüli kommunikáció

Fájlokon keresztüli kommunikáció

- Több processz ugyanabba a *fájlba ír, fájlból olvas*.
- Lassú, nagy átvitt információ-tartalmú. Lehet *FIFO csatorna* is.

Jellemző: indirekt, szimmetrikus, pufferelt, aszinkron, adat-küldéses, változó üzenetméretű kommunikáció.

- *Megvalósítás:* a I/O rendszerhívásokkal: *open()*, *read()*, *write()*, *close()*

IPC - vágólap `Clipboard()`

Vágólap - `Clipboard()`

- gyak. „ctrl+c ctrl+v” (Win, de más megvalósítás is létezik),
- közös memória terület, minden processz használhatja.

Mi is történik?

- Ctrl+C – aktuális processz ír a vágólapra: `SetClipboard*()`
- Ctrl+V – aktuális processz olvas a vágólapról
`GetClipboard*()`

IPC - vágólap `Clipboard()`

- De **nem** az OS része! Általában a GUI biztosítja.
- Indirekt, szimmetrikus, pufferezt, változó üzenethosszos kommunikáció.

IPC - environment

Az “**environment**”-en keresztüli kommunikáció:

- Az OS *környezeti változóiba* (environment variable) *írás*, ill. azok *kiolvasása* pl.: `akarmi=5; export akarmi; ... fork();`
- Gyors, kis információtartalom mehet át.
- Indirekt, zéró pufferelt, változó üzenethosszos, aszimmetrikus.

Megvalósítás: A burok környezeti változóit a processzek lekérdezhetik (*getenv()* rendszerhívás).

IPC mechanizmusok

Szignál

Related System Calls (System V)

Category	System Call	Description
General	open ()	This system call either opens an already existing file or creates and opens a new file.
General	creat ()	Creates and opens a new file.
General	read ()	Reads the contents of the file into the required buffer.
General	write ()	Writes the contents of buffer into the file.
General	close ()	Closes the file descriptor.
General	stat ()	Provides information on the file.
Pipes	pipe ()	Creates pipe for communication which returns two file descriptors for reading and writing.
Named Pipes or Fifo	mknod ()	Creates a memory device file or special file to create FIFOs
Named Pipes or Fifo	mkfifo ()	Creates a new FIFO
Shared Memory	shmget ()	Creates a new shared memory segment or gets the identifier of the existing segment.

Shared Memory	shmat ()	Attaches the shared memory segment and makes the segment a part of the virtual memory of the calling process.
Shared Memory	shmdt ()	Detaches the shared memory segment.
Shared Memory	shmctl ()	Performs control operations for the shared memory. Few of the generic control operations for the shared memory are removing the shared memory segment (IPC_RMID), receiving the information of the shared memory (IPC_STAT) and updating new values of the existing shared memory (IPC_SET).
Message Queues	msgget ()	Creates a new message queue or accesses an already existing message queue and gets the handle or identifier to perform operations with regard to message queue, such as sending message/s to queue and receiving message/s from the queue.
Message Queues	msgsnd ()	Sends a message to the required message queue with the required identification number.
Message Queues	msgrcv ()	Receives a message from the message queue. By default, this is infinite wait operation, means the call will be blocked until it receives a message.

Related System Calls (System V)

Message Queues	msgctl ()	Performs control operations for the message queue. Few of the generic control operations for the message queue are removing the message queue (IPC_RMID), receiving the information of the message queue (IPC_STAT) and updating new values of the existing message queue (IPC_SET).
Semaphores	semget ()	Creates a new semaphore or gets the identifier of the existing semaphore. Semaphores are used to perform synchronization between various IPCs working on the same object.
Semaphores	semop ()	Performs semaphore operations on semaphore values. The basic semaphore operations are either acquiring or releasing the lock on the semaphore.
Semaphores	semctl ()	Performs control operations for the semaphore. Few of the generic control operations for the semaphore are removing the semaphore (IPC_RMID), receiving the information of the semaphore (IPC_STAT) and updating new values of the existing semaphore (IPC_SET).
Signals	signal ()	Setting the disposition of the signal (signal number) and the signal handler. In other terms, registering the routine, which gets executed when that signal is raised.

Memory Mapping	mmap ()	Mapping files into the memory. Once mapped into the memory, accessing files is as easy as accessing data using addresses and also in this way, the call is not expensive as system calls.
Memory Mapping	munmap ()	Un-mapping the mapped files from the memory.

Forrás: tutorialspoint.com

System V & Posix

SYSTEM V	POSIX		
AT & T introduced (1983) three new forms of IPC facilities namely message queues, shared memory, and semaphores.	Portable Operating System Interface standards specified by IEEE to define application programming interface (API). POSIX covers all the three forms of IPC	Uses keys and identifiers to identify the IPC objects.	Uses names and file descriptors to identify IPC objects
SYSTEM V IPC covers all the IPC mechanisms viz., pipes, named pipes, message queues, signals, semaphores, and shared memory. It also covers socket and Unix Domain sockets.	Almost all the basic concepts are the same as System V. It only differs with the interface	NA	POSIX Message Queues can be monitored using select(), poll() and epoll APIs
Shared Memory Interface Calls shmget(), shmat(), shmdt(), shmctl()	Shared Memory Interface Calls shm_open(), mmap(), shm_unlink()	Offers msgctl() call	Provides functions (mq_getattr() and mq_setattr()) either to access or set attributes 11. IPC - System V & POSIX
Message Queue Interface Calls msgget(), msgsnd(), msgrcv(), msgctl()	Message Queue Interface Calls mq_open(), mq_send(), mq_receive(), mq_unlink()	NA	Multi-thread safe. Covers thread synchronization functions such as mutex locks, conditional variables, read-write locks, etc.
Semaphore Interface Calls semget(), semop(), semctl()	Semaphore Interface Calls Named Semaphores sem_open(), sem_close(), sem_unlink(), sem_post(), sem_wait(), sem_trywait(), sem_timedwait(), sem_getvalue() Unnamed or Memory based semaphores sem_init(), sem_post(), sem_wait(), sem_getvalue(),sem_destroy()	NA	Offers few notification features for message queues (such as mq_notify())
		Requires system calls such as shmctl(), commands (ipcs, ipcrm) to perform status/control operations.	Shared memory objects can be examined and manipulated using system calls such as fstat(), fchmod()
		The size of a System V shared memory segment is fixed at the time of creation (via shmget())	We can use ftruncate() to adjust the size of the underlying object, and then re-create the mapping using munmap() and mmap() (or the Linux-specific mremap())

Szignálok - jelzések

Egy *processz* számára a *szignál* egy ***esemény bekövetkeztét jelzi***, egyfajta információtartalom nélküli jelzés.

A *szignál kezelésére* a programot fel kell készíteni.

„A *szignálok* két csoportba sorolhatók:

- *elkapható*: fel lehet készíteni mi történjen a *szignál érkezésekor*, át lehet definiálni a *default* jelentést egy *szignálkezelő eljárás* segítségével.

Szignálok - jelzések

- *nem elkapható szignálok*: szignál érkezésekor azonnal aktiválódik a *default* jelentés, nem *lehet késleltetni*.

Felhasználása:

- *processzek kezelése* – mi ezzel foglalkozunk,
- a program működésének következményei, pl.: a saját memóriaterületén kívülre akar írni.
- egy kommunikációs vonal megszakadása.

Szignálok - jelzések

A szignálok célja:

- **értesítés** a *rendszer* és más *processzek* *eseményeiről*.

Szignál típusok (SIGINT, SIGHUP, SIGCHLD, SIGFPE, SIGKILL, ...)

- *rendszer*: kivételek (pl. hibák), kvóta, riasztás, értesítés (gyerek leállt),
- *felhasználói*: emberek (ctrl + c), processzek (tetszőleges céllal)

Szignálok - jelzések

Megvalósítás:

- *rendszerhívással* jelzést - értesítjük a kernelt,
- a kernel *értesíti a címzetteket* a jelzésről,
- a *címzett egy szignálkezelő eljárásában fogadja a jelzést.*

Szignálok - jelzések

A GNU C könyvtár (glibc) számos signal típust definiál, mindegyiket meghatározott eseményre.

- *Egyes signal-ok olyan végzetes hibákat, eseményeket jelentenek, ezek hatására a program alapértelmezés szerint kilép.*

Pl.: Ctrl+C-t nyomunk, vagy a program nullával oszt.

Szignálok - jelzések

- *Mások, kevésbé fontos eseményekről értesítenek, egyszerűen figyelmen kívül vannak hagyva, hatásukra semmi nem történik.*

P.: *signal*-t vált ki, ha egy *child processz* kilép.

Szignálok - jelzések

Mi is történik egy *esemény bekövetkezésekor*?

Ekkor definiálni kell *egy kezelőfüggvényt* az esemény által kiváltott *signal*-ra.

- Ekkor a rendszer a *signal érkezésekor* lefuttatja a ún. *Signal Handler*.
- Egy processz küldeni tud *signal*-t egy másiknak.

Szignálok - jelzések

Az események három kategóriába sorolható:

- *Hibák:* a program valami *érvénytelen dolgot művelt*,
- *Külső események:* lehetnek I/O események, vagy más processzek által kiváltottak. Pl.: *input érkezése* vagy egy *gyermekfolyamat kilépése*.
- *Közvetett kérések:* ilyen `KILL` függvény, mely egy kilépésre felszólító *signal*-t küld.

Szignálok - jelzések

Fontosabb szignálok:

KILL: *nem elkapható, hatására a processz azonnal befejeződik.*

TERM: *egy processzt befejezés.*

INT: *megszakítja a processz futását és a program leáll, azonban elkapható és átdefiniálható, pl. CTRL+Z -t esetén.*

Szignálok - jelzések

STOP : *megállítja a processz futását, így a processz várakozó állapotba kerül.*

CONT : a processz újra *futásra kész állapotúvá* tesz, így az bekerül az *ütemezési sorba* , tehát folytatja a felfüggesztett processzt.

ALRM : *Időzítő lejártáról* értesíti a processzt.

Szignálok - jelzések

CHILD: *egy gyermek processz megállításáról vagy befejeződéséről értesíti a szülő processzt.*

SIGUSR1 és SIGUSR2: a programozó szabadon definiálhatja a jelentését, bármire használhatjuk őket. Pl.: IPC SIGHUP: megszakadt a kapcsolat a terminállal (pl. hálózati hiba)

SIGPIPE: olyan *pipe*-ra írt, aminek nincsen olvasó processze.

Szignálok kezelése - kezelő rutin

Hogyan kezeljük le a kapott `signal`-t?

Írni kell egy *kezelő rutint* (handler) - és ezt hozzárendeljük a *signal* típusához/számához.

Ezt a `signal()` rendszerhívással tudjuk megtenni.

Tehát, a `signal()` hívásnak meg kell majd adnunk, hogy melyik `signal`-hoz szeretnénk hozzárendelni egy *kezelő rutint*,

Szignálok kezelése - kezelő rutin

A C programokban a szignálokra nevesített konstansokkal lehet hivatkozni. Pl: SIGINT (2), SIGKILL (9), SIGALRM (14), stb.

Két makro van definiálva, arra az esetre ha:

- figyelmen kívül szeretnénk hagyni a kapott signal-t (ignoráljuk), ekkor a függvény címnek a `SIG_IGN` – adjuk meg.
- az alap (default) kiszolgáló rutint szeretnénk újból hozzárendelni, ekkor a függvény címnek a `SIG_DFL` – adjuk meg.

Szignálok kezelése - signal()

Szignálok kezelése: a `signal()` függvény biztosítja a jelzések programból történő kezelését.

Szintaxis: - ez a régebbi változat

```
#include <signal.h>

int (*signal (jelzes, fuggveny)) ();
int jelzes;
int (*fuggveny) ();
```

vagy

Szignálok kezelése - signal()

```
#include <signal.h>

void (*signal (int jelzes, void
(*fuggveny) (int))) (int);
```

- az argumentumok *típusa* is fel van tüntetve: *egy egész szám* és *egy mutató egy függvényhez*.
- A *jelzes paraméter* a *jel* száma, amelynek a kezelését szeretnénk testreszabni.

Szignálok kezelése - signal()

A függvény argumentum leírja, hogyan kezelje a processz az adott jelet.

A következő értékeket veheti fel:

- `SIG_DFL` – default kezelés , a jelzések kezelését a rendszer magja végzi - ez a jelzések *implicit kezelése*.
- `SIG_IGN` – figyelmen kívül hagyja az adott jelet (kivétel a `SIGKILL` és a `SIGSTOP`).

Szignálok kezelése - signal()

Hiba esetén a `SIG_ERR` konstanst kapjuk, amelynek definíciója:

```
#define SIG_ERR (void (*)( )) -1;
```

Szignálok kezelése – sighandler létrehozása

Szignálok kezelése – sighandler létrehozása

```
#include <signal.h>
```

```
sighandler_t signal(int signum, sighandler_t handler);
```

A signal függvénynek két paramétere van:

- az első egy (általában nevesített konstanssal) *szignál azonosító/szám*,
- második paraméter a korábban megírt *szignálkezelő eljárás neve, címe*.

Szignálok kezelése - `kill`, `raise`()

`kill`, `raise`

A `kill` függvény egy *jelet küld egy processznek vagy egy processz csoportnak* - POSIX.1 verzióban van definiálva.

A `raise` megengedi a processznek, hogy *saját magának is küldjön jelet* - ANSI C-ben.

További információk:

Lásd: man 2

Szignálok kezelése - mintapél

- Írjon C nyelvű programot, amelyik a SIGTERM-hez hozzárendel egy fv-t., amelyik kiírja az int paraméter értéket.
- Majd végtelen ciklusban fusson, 3 sec-ig állandóan blokkolódva elindítás után egy másik shell-ben kill paranccsal (SIGTERM) próbálja terminálni, majd SIGKILL-el.”

Szignálok kezelése - `kill`, `raise()`

Szintaxis:

```
#include <sys/types.h>
#include <signal.h>

int kill(pid_t pid, int jel);
int raise(int jel);
```

Ha a művelet sikeres volt, mindkét processz 0 értékkel tér vissza, és -1-et hiba esetén.

A `pid` változó azt a processzt/processz csoportot jelöli, aminek a `jel` jelet szeretnénk küldeni.

Szignálok kezelése - `kill`, `raise()`

A `raise` függvény implementációja a `kill` függvény segítségével:

```
#include <sys/types.h>
#include <signal.h>
#include <unistd.h>

int raise(int jel)
{
    return (kill(getpid(), jel));
}
```

Szignálok kezelése `-kill, raise()`

Signál küldésre használható hívások: `kill()` `raise()`

Leírás: `man 2 kill;` `man 3 raise`

A `kill()` a megadott processznek küld egy megadott `signal`-t.

Lásd: <https://man7.org/linux/man-pages/man2/kill.2.html>

Szignálok kezelése -leírás

Linux rendszerbe pl. a `header` állományban találjuk meg,
hogy milyen típusú szignálok vannak definiálva:

```
/usr/include/bits/signum.h
```


Szignálok kezelése – `alarm`, `pause`

`alarm`, `pause`

Az `alarm` függvény lehetővé teszi egy *időzítő beállítását*.

A megadott idő (mp) elteltével egy `SIGALRM` jelet ad ki.

Alakja:

```
#include <unistd.h>
unsigned int alarm(unsigned int mp)
```

A visszatérési érték vagy 0, vagy az előző `SIGALRM` jelzés kiadása óta eltelt másodpercek száma.

Szignálok kezelése - `alarm`, `pause`

A `pause` függvény felfüggeszti (blockod) a hívó processzt a legelső jel érkezéséig.

```
#include <unistd.h>
int pause(void);
```

A `pause` függvényt legtöbbször az `alarm`-mal együttesen szoktuk használni.

IPC - csővezeték

Csővezeték

IPC - csővezeték - pipe()

Egy létrehozott *csőbe ír* és a *fogadó onnan olvas*.

A *cső FIFO jellegű* és amit kiolvasnak az *törlődik* (lehetséges két csővel szimmetrikus kapcsolat is).

Lehet: *Nevesített és névnélküli* cső mechanizmusok.

- *Nevesített csövek (named)* megvalósítása fájlokon keresztüli kommunikáció: lassú sebességű, nagy átvitt adatmennyiség, indirekt.

IPC - csővezeték - pipe()

- *Névtelen csövek (unnamed) megvalósítása rendszerfüggő.*

Lehet fájlokon keresztüli, lehet osztott memórián vagy egyéb módon. Ettől függően más és más jellemzők.

- Aszimmetrikus, változó üzenethosszos kommunikáció.

IPC – Csővezeték - pipe() – névnélküli cső

Def.: A csővezeték (pipe) a rendszer által létrehozott (névnélküli) speciális állomány.

Maximális hossza 10 blokk (azaz 5120 byte).

Feladata: két procesz közötti kommunikáció megvalósítása úgy történik, hogy az

- *egyik processz ír a standard kimenetre,*
- *a másik, pedig olvas a standard bemenetről.*

IPC - Csővezeték - pipe()

A pipe FIFO szerkezetű (elsőnek be, elsőnek ki).

Két mutatója van: egyik az író folyamaté, a másik az olvasóé - így az írás/olvasás cirkulárisan történik.

Ha a pipe megtelik az írónak kell várnia, ha kiürül az olvasónak. A kiolvasott adatok törlődnek a listából.

A `pipe` fájlokat csak *apa-fiú kapcsolatban* lévő (vagy közös őssel) processzek használhatnak.

IPC - Csővezeték - pipe()

A *pipe* fájlok szintjén a következő műveleteket kell (lehet) elvégezni:

- a) az *állomány létrehozása* a `pipe()` függvény segítségével,
- b)** *olvasás/írás* a `read/write()` függvények segítségével,
- c)** az *állomány bezárása* a `close()` függvény segítségével,
- d)** az állomány *törlése*.

IPC - Csővezeték (FIFO)

Csővezeték (FIFO) – Nevesített csővezeték

IPC - Csővezeték (FIFO) – Nevesített csővezeték

Def.: olyan állomány, amelyhez *bármelyik processz hozzáférhet*, amelynek van rá megfelelő joga...

A névvel ellátott csővezeték (FIFO állomány) a közönséges fájl és a pipe kombinációja.

A FIFO állománynak van egy szimbolikus neve, és egy könyvtára, ahová létrehozzuk.

Megőrzi a *pipe* fájlok összes jellemzőit.

IPC - Csővezeték (FIFO) – Nevesített csővezeték

- *A nevesített csővezeték előnye, hogy egy FIFO fájlt bármely processz megnyithat írásra, ill. olvasásra egy - csak a szimbolikus nevet kell ismernie.*
- Egy FIFO állomány megtekinthető az `ls -l` Unix parancs segítségével.
- A hozzáférési jogok első mezője ez esetben egy `p` betű.

IPC - Üzenetsoros kommunikáció

Üzenetsoros kommunikáció

IPC – üzenet soros kommunikáció

Def.: Processzek közti kommunikáció *üzenetek* (kis adatcsomagok) formában történik.

Az egyik *processz üzenetsort készít*, más processzek kapcsolódnak rá és *sorba betehető* és az *kivehető információ*.

- Az üzenetsor *processzekről függetlenül* létezik.
- *Tulajdonossági és hozzáférési jellemzőiket* a fájl tulajdonossági, hozzáférési kategóriák segítségével szabályozzák.

IPC – üzenet soros kommunikáció

- Gyors, közepes átvihető információmennyiség.
- Indirekt, OS kötődésű, többirányú, végtelen puffer kapacitású, változó üzenethosszú.

IPC – messages

„Az üzenetsor egy láncolt lista, amelynek jól meghatározott azonosítója van.

- Egy processz az *üzenetsort* egy *kulcs* segítségével *azonosít*.
- A *kulcsot* mi választjuk - az *azonosítót*, pedig a rendszer minden *üzenetsorhoz automatikusan* rendeli hozzá.

IPC – messages

- Ha üzenetet szeretne *küldeni*, előbb az *adott kulcs alapján* az `msgget` függvénnnyel lekéri az *üzenetsor azonosítóját*, majd az `msgsnd` függvény segítségével *elküldi az üzenetet* - az üzenet a sor végére kerül.
- Ha üzenetet szeretne *fogadni*, szintén lekérdezi *a kulcs és* az `msgget` függvény segítségével az *üzenetsor azonosítóját*, majd az `msgrcv` függvénnnyel *kiolvassa az üzenetet*.

IPC – messages

- *Az üzenetsorból való olvasás nem feltétlenül a FIFO módszer szerint történik, ugyanis az `mt_type` mező értékétől függően az üzeneteket *tetszőleges sorrendben is kiolvashatjuk.**

IPC mechanizmusok

Osztott memórián keresztüli kommunikáció

IPC - osztott memória

Def.: Az osztott memória segítségével megoldható, hogy két vagy több processz ugyanazt a memóriarészt használja.

- Gyors, nagy átvihető információ mennyiség.
- Indirekt, szimmetrikus, zero pufferelt, változó üzenethosszos.

IPC - osztott memória

Az osztott memória kommunikáció elvei:

- Egy *processz* létrehoz egy *közös memóriarészt*.
- A *processz azonosítója* bekerül a *memóriarészbe* rendelt *struktúrába*.
- A létrehozó processz *hozzarendel az osztott memóriához egy numerikus kulcsot* – a többi processznek ismerni kell.
- A *memóriarészt* az `shm_id` változó azonosítja.

IPC - osztott memória

- A létrehozó *processz hozzárendeli többi processz hozzáférési jogait.*
- Ez a kommunikáció a leggyorsabb, azért mert az adatokat nem kell mozgatni a *kliens* és a *szerver* között.

IPC - osztott memória - műveletek

Műveletek *az osztott memóriával:*

- létrehozás - `shmget()`,
- osztott memória *adatainak lekérdezése, módosítása és törlése* - `shmctl()`,
- memóriarész *hozzárendelése* `shmat()`,
- memóriarész *hozzárendelésének megszüntetése* `shmdt()`.

IPC mechanizmusok - Szemaforok

Szemaforok

IPC - szemafor

Szemaforok, mint kommunikációs mechanizmusok

A szinkronizáció alapvető eszköze.

Van két állapotú, vagy több állapotú szemafor.

(hasonló a vonatokat vezérlő szemafor elgondoláshoz)

- Gyors, nagyon kicsi átvitt információ mennyiség
- Indirekt, kétirányú, zero pufferelt, fix hosszúságú.

IPC - Szemaforok

Megalkotása *Edsger Dijkstra* holland matematikusnak, nevéhez fűződik - az IPC egyik eszköze, valószínűleg egy vasutas ősoktól származott UNIX-os fogalom.

Miről is van itt szó?

Nézzünk meg egy példát erre!

IPC - Szemaforok

A vonat megáll egy *piros szemafor előtt*, és addig *várakozik*, amíg szabad utat nem kap, mert valamilyen oknál fogva (foglalt a pálya stb.) a továbbhaladás meg van tiltva.

Az OS- nél is hasonló. Feltételezzünk, hogy van egy *kritikus erőforrásunk*, amit egyszerre csak egy erőforrás használhat , kezdőértéke =1.

Amikor a *kritikus művelethez* érek, akkor **jelzem** az *erőforrás-használati igényemet*.

IPC - Szemaforok

P1 - A jelzés jelentse azt, hogy eggyel csökkentem a szám értékét.

Ezt szokás „down” vagy sok más helyen „P” operációnak (**P**robeer te verlagen, megpróbálni csökkenteni, holland kifejezés) is nevezni.

Ha a csökkentés eredmény nem negatív lesz, akkor szabad az út, és végzi a dolgát a program.

IPC - Szemaforok

Ha ezután érkezik egy *P2 processz*, ami ugyanezt az erőforrást szeretné használni, de neki már a **P** operáció pirosra állítja a „szemafort, mert az „egész” értéke mínusz 1 lesz.

Ekkor ez a *P2 processz* mindaddig vár, amíg az „egész”-szemafor értékét egy ún. „**UP**” vagy „**V**” operációval (Verhogen, növelni, holland szó) – ami az eggyel való növelést jelenti – fel nem szabadítja az erőforrást, ami után a P operációnál várakozó program tovább haladhat.

IPC - Szemaforok

Egy szemafor kezdőértéke nemcsak 1 lehet, hanem tetszőleges egész szám.

Ha a szemafor kezdőértéke 1, akkor *bináris szemafor*nak is nevezik, hiszen:

- ha 1 az értéke, akkor az a szabad jelzést,
- ha 0 az értéke, az a tilos jelzést jelenti,

azaz csak a 0 és 1 értékeket veheti fel.

IPC - Szemaforok

Def.: Olyan jelek, amelyek megmutatják, hogy egy processz végrehajthat-e egy programrészt vagy sem...

Cél: A processzor maximális kihasználása érdekében OS jelentős része engedélyezi a folyamatoknak a párhuzamos (egydőben történő) futást.

Kérdés: mi történik akkor, ha két folyamat egyidőben próbálja elérni ugyanazt az erőforrást? - pl.: nyomtató, memória.

IPC - Szemafor - Létrehozás `semget()`

Létrehozás - `semget()`

A `semget` rendszerfüggvény lehetővé teszi egy *szemaforköteg ID-jának* a meghatározását.

Ha a szemaforköteg előzőleg nem létezett, a függvény létrehozza azt.

A processznek ismernie kell a szemaforhoz *rendelt kulcsot*.

A Socket

A socket többfajta kommunikációt valósít meg egy OS belül két processz között, vagy számítógépes hálózaton keresztül két különböző számítógépen futó folyamatok között:

- küldhetünk egy irányban egy adatcsomagot,
- vagy nagy mennyiségű adatot is csomagokra bontva hibakezeléssel,
- párhuzamos kiszolgálást segítő függvényei is vannak.” [1][2][3][4]

Irodalom

1. Vadász Dénes: Operációs rendszerek, ME, 2007.

<http://www.iit.uni-miskolc.hu/~vadasz/GEIAL302B/2007-Ea>

2. Heizlerné Bakonyi Viktória, Horváth László, Illés Zoltán, Nagy Tibor, Lutár Patrícia: Számítógépes alapismeretek II. ELTE 2012.

https://regi.tankonyvtar.hu/hu/tartalom/tamop412A/2010-0011_szamalap2/adatok.html

Irodalom

3. Kóczy, Kondorossi szerk: Operációs rendszerek mérnöki megközelítésben, Panem, 2000.

4. Szemaforok (semaphores)

<https://www.cs.ubbcluj.ro/~robu/so2/folyamatok/szemafor.htm>